

Feedback recibido de la primera entrega.

- No aporta cálculo de combinatoria sin restricciones.
- La combinatoria con restricciones tampoco es del todo exacta porque puede haber un óptimo que no llene el cupo de uno de los días, según distribuciones de actores (aunque no es el caso de este dataset).
- Solventan con heurísticas greedy y aleatoria, que se indicó durante el curso que no puntuarían.
- La comparativa propuesta entre FB y heurística para un problema reducido no tiene sentido ya que la restricción por día es más laxa que el número de tomas a asignar.

Revisar estos aspectos y aportar una solución y comparativa algo más elaboradas. Y volver a entregar en 2a convocatoria.

Resumen de las modificaciones realizadas para la entrega número 2.

- Cálculo de combinatorio sin restricciones añadido.

$$30^{30} = 2058911320946490000000000000000000000000000000000$$

- Combinatoria con restricciones corregida.

$$P_{30}^{6,6,6,6,6} = \frac{30!}{6! \cdot 6! \cdot 6! \cdot 6! \cdot 6! \cdot 5!} = \frac{30!}{(6!)^5 \cdot 5!}$$

$$P = 11423951396577720$$

- Se han añadido dos algoritmos aparte del voraz y búsqueda aleatoria:
 - Búsqueda local con operador de intercambio (swap)
 - Recocido Simulado.
- Se ha corregido la comprativa propuesta para la fuerza bruta, ya que como se mencionaba, la restricción por registros era menos restrictiva que el número de tomas.
 - Antes: $n=5$ max_tomas_día= 6
 - Corrección: $n=8$ max_tomas_día= 2
- Por último, se ha añadido un tabla comparativa con los resultados de coste y tiempo, de todos los algoritmos usados.

Algoritmos de optimización - Seminario

Nombre y Apellidos: Ángel Encinas Moreno & Williams Mérida Parard

Url:<https://github.com/Williams->

[MP/Algoritmos_de_optimizacion_entregas/tree/9086b65de7cbc459251d3467e8d2877842f](https://github.com/Williams-MP/Algoritmos_de_optimizacion_entregas/tree/9086b65de7cbc459251d3467e8d2877842f)

Problema:

1. Sesiones de doblaje

Descripción del problema:

Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible. Los datos son:

Número de actores: 10 Número de tomas : 30 Actores/Tomas : <https://bit.ly/36D8luK>

(*) La respuesta es obligatoria

Uso de la IA

Para la obtención de los bloques de código nos hemos apoyado en IA generativa, tanto para optimizar ideas propias como para solucionar errores persistentes que encontrábamos en el camino. En ningún momento se ha generado desde cero y simplemente copiado y pegado ningún bloque de código.

Importación de librerías necesarias

```
In [1]: import pandas as pd
import numpy as np
import time
import math
import random
import itertools
```

Carga de datos originales

```
In [2]: # Cargamos el archivo excel en la misma ruta.
nombre_archivo = "Datos problema doblaje(30 tomas, 10 actores).xlsx"
# Ajusta el skiprows o usecols si tu Excel tiene cabeceras raras
df_completo = pd.read_excel(nombre_archivo)

# Seleccionar los datos numéricos y evitamos las cabeceras
datos_numericos = df_completo.iloc[1:31, 1:11].dropna().astype(int)

# Creamos un dataframe con los datos
registro_sesiones = pd.DataFrame(
    datos_numericos.values,
    index=np.arange(len(datos_numericos)),
    columns=['Actor ' + str(x) for x in np.arange(1, 11)]
)
registro_sesiones.index.name = 'Toma -1 '

# Calculamos el total, con esto aseguramos cualquier discrepancia con el total
registro_sesiones['Total'] = np.sum(registro_sesiones, axis=1)

# Visualizamos los resultados
display(registro_sesiones.head())

# Asignamos el DF a la variable data.
data = registro_sesiones
```

	Actor 1	Actor 2	Actor 3	Actor 4	Actor 5	Actor 6	Actor 7	Actor 8	Actor 9	Actor 10	Total
Toma -1											
0	1	1	1	1	1	0	0	0	0	0	5
1	0	0	1	1	1	0	0	0	0	0	3
2	0	1	0	0	1	0	1	0	0	0	3
3	1	1	0	0	0	0	1	1	0	0	4
4	0	1	0	1	0	0	0	1	0	0	3

Dataframe con valores aleatorios

Más adelante se solicitará la creación de una muestra de datos aleatorias. En nuestro caso hemos decidido, crearla ahora, para poder ser utilizada en todos los algoritmos presentados en este proyecto. Los valores aleatorios siguen una distribución uniforme, que lo se verá reflejado en el coste cuando se muestren los resultados.

```
In [3]: def generar_datos_aleatorios_uniformes(n_tomas, n_actores):
        """
        Genera una matriz de ceros y unos totalmente aleatoria.
        """
        # Crear matriz de n_tomas x n_actores con 0 y 1
```

Nuevo set de datos generado aleatorios con distribución uniforme:

	Actor 1	Actor 2	Actor 3	Actor 4	Actor 5	Actor 6	Actor 7	Actor 8	Actor 9	Actor 10	Total
Toma-1	0	0	1	1	1	1	0	0	0	0	4
1	0	0	0	0	0	0	1	1	0	0	2
2	1	0	1	0	0	0	1	0	0	1	4
3	0	1	0	1	1	0	1	1	0	0	5
4	0	1	1	1	0	0	1	1	1	1	7

¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

Si no tenemos en cuenta las restricciones de capacidad y tiempo se podría asegurar que las posibilidades serían virtualmente infinitas. Al no existir un número mínimo de tomas por día, ni un máximo de días para finalizar la grabación, el número de calendarios de rodaje no tiene un límite acotado, ya que se podría extender la producción indefinidamente sin llegar a completar nunca las grabaciones.

Por otra parte considerando la lógica básica del enunciado, donde al menos una toma ha de ser grabada al día, entonces tenemos un universo que va desde:

- 1 día: donde en un día se grabarían todas las tomas
- 30 días: donde se grabaría una toma al día
- Para este caso el total de combinaciones ascendería a

$$30^{30} = 2058911320946490000000000000000000000000000000000$$

Modelo para el espacio de soluciones

(*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo. (Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

- Para representar esto de la mejor forma, la estructura de datos más adecuada es una lista de números enteros de tamaño 30. Cada posición n de la lista corresponde a la toma n de nuestra tabla de datos, y el número almacenado en esa posición representa el día que se le ha asignado.
 - Siguiendo las restricciones mencionadas, cualquier lista que sea una solución válida será una permutación del conjunto A . De esta manera, nos aseguramos de que cada día (del 1 al 5) aparezca exactamente 6 veces en la lista, respetando el límite de capacidad y cubriendo todas las tomas.
-

Según el modelo para el espacio de soluciones

(*)¿Cual es la función objetivo?

- Se puede decir que la función objetivo es aquella que calcula el número de desplazamientos a la grabación de los actores. Buscamos contabilizar el número de veces que aparece cada actor de doblaje a lo largo de todos los días de grabación.
- Para obtener la función hay que agrupar las tomas asignadas a cada día.
 - Cada toma se representa con dos valores (binario), donde 1 y 0 representan "asistir" y "no asistir" respectivamente.
 - Debido a que un actor ha de ir al estudio independientemente de si graba una toma o más de 1, entonces se ha de incluir la función lógica **OR**. Esta se puede representar con $|$.
 - Finalmente, deberíamos contar todos los actores que han ido ese día al estudio. Es decir, sumamos los actores. Sería equivalente a la función `count()`.
 - Con esta información podemos decir que función objetivo sería algo como:

$$\sum_{j=1}^M count(T_{1j}|T_{2j}|\dots|T_{kj})$$

donde: j = índice del día, M : total de días de duración de la grabación,

$$T_{ij}$$

donde i = el número de toma y j = el día asignado

(*)¿Es un problema de maximización o minimización?

Es un problema de minimización. Se intenta reducir al máximo la cantidad de veces que los actores van al estudio, agrupando las tomas de forma que los repartos coincidan lo más posible, y de esta manera reducir costes.

Ejemplo

Vamos a usar un ejemplo para que se vea la idea. Imaginemos que tenemos 4 tomas y 2 días disponibles (máximo 2 tomas por día).

- **Solución 1:** [1,1,2,2] Agrupamos las tomas 1 y 2 el Día 1, y las tomas 3 y 4 el Día 2.
 - Día 1 (Tomas 1 y 2): Imaginemos que el OR lógico da 5 actores.
 - Día 2 (Tomas 3 y 4): Imaginemos que el OR lógico da 4 actores.
 - Total pagas: $5 + 4 = 9$
- **Solución 2:** [1,2,1,2] Agrupamos las tomas 1 y 3 el Día 1, y las tomas 2 y 4 el Día 2.
 - Día 1 (Tomas 1 y 3): Imaginemos que el OR lógico da 6 actores.
 - Día 2 (Tomas 2 y 4): Imaginemos que el OR lógico da 6 actores.
 - Total pagas: $6 + 6 = 12$
- **Conclusión:**
 - La Solución 1 vuelve a ser mejor (9 que pagamos contra 12). Se puede ver que al mezclar tomas que no comparten actores (Solución 2), aumentamos innecesariamente el número de personas que deben desplazarse al estudio cada día.

Para encontrar la mejor combinación de rodaje entre todas las posibilidades, primero necesitamos una herramienta matemática que nos diga exactamente cuánto cuesta cualquier plan que se nos ocurra. La función **calcular_coste_rodaje** es nuestro motor de evaluación: toma un plan de trabajo, aplica la lógica de asistencia de los actores y nos devuelve el gasto total en dinero.

```
In [4]: # Es función lo que hace es sumar cuánto nos va a costar el rodaje
# según cómo hayamos repartido las tomas en los días.
def calcular_coste_rodaje(plan_rodaje, matriz_asistencia, verbose=True):
    try:
        # Primero comprobamos que hayamos asignado un día a cada toma
        assert len(plan_rodaje) == len(matriz_asistencia)
    except:
        print('CUIDADO: Las tomas y el plan de días no coinciden')
        return float('inf')

    # Hacemos una copia de los datos por seguridad
    datos_con_dia = matriz_asistencia.copy()
    # Añadimos una columna nueva para saber en qué día se graba cada toma
    datos_con_dia['DiaAsignado'] = pd.Series(plan_rodaje)
```

```

total_pagas = 0
# Vamos recorriendo cada día de rodaje
for dia_actual in np.unique(plan_rodaje):
    # Seleccionamos solo las tomas que tocan ese día
    tomas_del_dia = datos_con_dia[datos_con_dia['DiaAsignado'] == dia_actual]

    # tomas_del_dia.iloc[:, :-2] --> cogemos los 1s y 0s de los actores
    # np.any(, axis=0), --> Si un actor sale en alguna toma del día se marca
    # 3. np.sum() --> Contamos cuántos actores tienen que venir, o sea los "
    actores_que_vienen = np.sum(np.any(tomas_del_dia.iloc[:, :-2], axis=0))
    total_pagas += actores_que_vienen

if verbose: print(f'Total pagas con este plan {plan_rodaje}: {total_pagas}')
return total_pagas

```

Diseña un algoritmo para resolver el problema por fuerza bruta

La estrategia aquí es simple pero costosa: generamos todas las combinaciones posibles de tomas por día, calculamos el coste de cada una y nos quedamos con la más barata. Para esto, necesitamos crear una función que primero calcule todas las permutaciones y luego vaya probando una a una hasta dar con la solución óptima.

```

In [5]: def fuerza_bruta(matriz_asistencia, max_tomas_dia):
    total_tomas = len(matriz_asistencia)
    # Calculamos el mínimo de días teóricos
    dias_minimos = math.ceil(total_tomas / max_tomas_dia)

    # Pintamos un poco de info inicial
    print(f'Actores totales en la grabación: {len(matriz_asistencia.loc[0])-1}')
    print(f'Tomas a grabar: {total_tomas} ')
    print(f'Límite de tomas al día: {max_tomas_dia} -> mínimo {dias_minimos} c

    # Creamos una lista base repartiendo las tomas entre los días mínimos
    # Ejemplo con 6 tomas y 3 por día -> [1, 1, 1, 2, 2, 2]
    plantilla_base = [dia for dia in range(1, dias_minimos + 1) for _ in range
    plantilla_base = plantilla_base[:total_tomas]

    # Generamos TODAS las permutaciones (esto es lo que tarda tanto)
    todas_las_combinaciones = itertools.permutations(plantilla_base)

    coste_minimo = float('inf')
    mejor_plan = []

    # Vamos probando cada combinación una por una
    for plan in todas_las_combinaciones:
        # Calculamos cuánto cuesta este plan usando la función que hicimos antes
        coste_actual = calcular_coste_rodaje(plan, matriz_asistencia, verbose=False)

        # Si este plan es más barato que el anterior más barato, nos lo quedamos
        if coste_actual < coste_minimo:

```



```

coste_minimo = coste_actual
mejor_plan = plan

print(f'Mejor asignación: {mejor_plan}, con coste {coste_minimo}')
return mejor_plan, coste_minimo

```

Calcula la complejidad del algoritmo por fuerza bruta

La complejidad es:

$$\frac{n!}{k!^M}$$

Este es principal inconveniente de este método, donde el número de combinaciones puede incrementar muy rápidamente. Como esto crece a un ritmo de factorial $O(n!)$. Para nuestro caso de 30 tomas, no será posible probar todas las posibilidades, a no ser que lo hagamos por computación cuántica.

IMPORTANTE Debido a que la complejidad es factorial, no será posible obtener un resultado usando 30 registros. Para comprobar que la función trabaja correctamente, reduciré a 5 registros (tomas).

Aplica el algoritmo Fuerza Bruta al juego de datos original reducido

```

In [6]: # Creamos una variable 'data_para_fuerza_bruta' con solo 8 tomas y un límite
# Para comprobar que el código funciona con menos registros.
# Seleccionamos las primeras 5 filas (iloc[0:8])
data_para_fuerza_bruta = data.iloc[0:8].copy()

```

```

In [7]: # Ponemos el cronómetro
inicio_cronometro = time.time()

# Ejecutamos la función fuerza bruta
_, mejor_coste_FB_dataset_original = fuerza_bruta(data_para_fuerza_bruta, 2)

# Calculamos la duración.
tiempo_total_FB_dataset_original = round(time.time() - inicio_cronometro, 2)
print(f'Búsqueda Óptima: coste {mejor_coste_FB_dataset_original} en {tiempo_

```

Actores totales en la grabación: 10

Tomas a grabar: 8

Límite de tomas al día: 2 -> mínimo 4 días

Mejor asignación: (1, 1, 2, 2, 3, 4, 4, 3), con coste 19

Búsqueda Óptima: coste 19 en 108.96 segundos

Aplica el algoritmo Fuerza Bruta al juego de datos aleatorios reducido generado

```
In [8]: # Creamos una variable 'data_para_fuerza_bruta' con solo 8 tomas y un límite
# Para comprobar que el código funciona con menos registros.
# Seleccionamos las primeras 5 filas (iloc[0:8])
datos_aleatorios_para_fuerza_fruta = df_aleatorio.iloc[0:8].copy()
```

```
In [9]: # Ponemos el cronómetro
inicio_cronometro = time.time()

# Ejecutamos la función fuerza bruta
_, mejor_coste_FB_dataset_aleatorio = fuerza_bruta(datos_aleatorios_para_fue

# Calculamos la duración.
tiempo_total_FB_dataset_aleatorio = round(time.time() - inicio_cronometro, 2)
print(f'Búsqueda Óptima: coste {mejor_coste_FB_dataset_aleatorio} en {tiempo
```

Actores totales en la grabación: 10

Tomas a grabar: 8

Límite de tomas al día: 2 -> mínimo 4 días

Mejor asignación: (1, 2, 3, 4, 4, 2, 1, 3), con coste 20

Búsqueda Óptima: coste 20 en 132.83 segundos

(*)Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Voraz

Tras comprobar que la fuerza bruta es inviable ($O(n!)$), una posible alternativa es implementar un algoritmo voraz. Esta técnica es ideal cuando buscamos eficiencia y el espacio de soluciones es muy grande.

¿Por qué elegir esta técnica?

- Según la definición de clase, la técnica voraz se basa en dividir el problema por etapas, tomando en cada una la decisión más ambiciosa y óptima sin mirar atrás.
 - **Eficiencia:** Es la técnica más recomendable por su rapidez.
 - **Aplicabilidad:** Cumple con las características necesarias: tenemos candidatos (tomas), una función de selección (agrupar por proximidad de actores), y una función de factibilidad (máximo k tomas por día).

Funcionamiento Aplicado al Problema:

- Selección: Localizamos la toma con más actores y asignamos el día.
- Proximidad: Calculamos cuántos actores comparten las demás tomas con la elegida.
- Ambición: Agrupamos las tomas con mayor proximidad para minimizar costes inmediatamente. Conclusión: Aunque voraz no garantiza el óptimo global en todos

los casos, nos proporciona una solución factible en un tiempo menor, superando el mayor inconveniente de la fuerza bruta.

```
In [10]: # Este algoritmo construye la solución día a día, toma la mejor decisión local
def alg_voraz(matriz_asistencia, max_tomas_dia, verbose=True):

    # --- Complejidad en área 1: Preparación [O(n * m)] ---

    # Trabajamos sobre una copia para no romper los datos originales
    datos_trabajo = matriz_asistencia.copy()
    total_tomas = len(datos_trabajo)
    dias_totales = math.ceil(total_tomas / max_tomas_dia)

    if verbose:
        print(f'Actores totales en la grabación: {len(datos_trabajo.loc[0])}-')
        print(f'Tomas a grabar: {total_tomas}')
        print(f'Límite de tomas al día: {max_tomas_dia} -> mínimo {dias_totales}')

    # Array para guardar qué día le toca a cada toma
    plan_final = np.zeros(total_tomas, dtype=int)

    # --- Complejidad en área 2: Bucle principal [Se repite n/k veces] ---
    # Vamos rellenando los días uno por uno
    for dia_actual in range(1, dias_totales + 1):
        scores_parecidos = np.zeros(len(datos_trabajo))

        # Buscamos la toma con más actores para que sea nuestra referencia
        indice_toma_ancla = datos_trabajo['Total'].idxmax()
        plan_final[indice_toma_ancla] = dia_actual
        # Marcamos esta toma para no volver a usarla
        scores_parecidos[datos_trabajo.index.get_loc(indice_toma_ancla)] = -1

    # --- Complejidad en área 3: Compatibilidad [O(n * m)] ---

    # Calculamos cuántos actores comparten las demás tomas con la referencia
    for posicion, indice_fila in enumerate(datos_trabajo.index):
        if indice_fila != indice_toma_ancla:
            # np.all comprueba qué actores coinciden en las tomas
            scores_parecidos[posicion] = np.sum(np.all(datos_trabajo.loc[indice_fila, :] == datos_trabajo.loc[indice_toma_ancla, :]))

    # --- Complejidad en área 4: Ordenación [O(n log n)] ---

    # Ordenamos las tomas por compatibilidad y cogemos las mejores
    datos_trabajo['Compatibilidad'] = scores_parecidos
    tomas_ordenadas = datos_trabajo.sort_values('Compatibilidad', ascending=False)

    # Seleccionamos las k-1 tomas más parecidas para agruparlas
    tomas_a_agrupar = tomas_ordenadas[:min(max_tomas_dia - 1, len(tomas_ordenadas))]
    plan_final[tomas_a_agrupar] = dia_actual

    # --- Complejidad en área 5: Actualización [O(n)] ---

    # Quitamos las tomas ya asignadas de la lista de pendientes
    datos_trabajo.drop(tomas_a_agrupar, axis=0, inplace=True)
    datos_trabajo.drop(indice_toma_ancla, axis=0, inplace=True)
```

```
# ---Complejidad en área 6: Evaluación final [O(n * m)] ---

# Convertimos a lista y calculamos el coste final
plan_final = plan_final.tolist()
coste_final = calcular_coste_rodaje(plan_final, matriz_asistencia, verbose)

if verbose: print(f'Solución voraz con coste {coste_final}: {plan_final}')
return plan_final, coste_final
```

Aplica el algoritmo Voraz al juego de datos original

```
In [11]: # Cronómetro
inicio_voraz = time.time()

# Ejecutamos la función voraz
_, coste_voraz_dataset_original = alg_voraz(data, 6, verbose=True)

# Calculamos y pintamos el tiempo y el resultado
tiempo_voraz_dataset_original = round(time.time() - inicio_voraz, 2)
print(f'Estrategia Voraz: coste {coste_voraz_dataset_original} en {tiempo_vo
```

Actores totales en la grabación: 10

Tomas a grabar: 30

Límite de tomas al día: 6 -> mínimo 5 días

Solución voraz con coste 35: [1, 3, 2, 2, 2, 1, 1, 3, 2, 2, 1, 1, 3, 4, 2, 5, 3, 5, 4, 1, 5, 3, 4, 5, 3, 4, 5, 4, 4, 5]

Estrategia Voraz: coste 35 en 0.11 segundos

Aplica el algoritmo Voraz al juego de datos aleatorios generado

```
In [12]: # Cronómetro
inicio_voraz = time.time()

# Ejecutamos la función voraz
_, coste_voraz_dataset_aleatoria = alg_voraz(df_aleatorio, 6, verbose=True)

# Calculamos y pintamos el tiempo y el resultado
tiempo_voraz_dataset_aleatoria = round(time.time() - inicio_voraz, 2)
print(f'Estrategia Voraz: coste {coste_voraz_dataset_aleatoria} en {tiempo_v
```

Actores totales en la grabación: 10

Tomas a grabar: 30

Límite de tomas al día: 6 -> mínimo 5 días

Solución voraz con coste 44: [2, 4, 2, 1, 1, 5, 5, 4, 4, 4, 3, 5, 1, 1, 3, 5, 3, 4, 2, 5, 3, 3, 4, 2, 3, 1, 1, 2, 2, 5]

Estrategia Voraz: coste 44 en 0.12 segundos

(*)Calcula la complejidad del algoritmo

Variables:

- n : Número total de tomas.
- m : Número de actores (columnas).
- k : Máximo de tomas por día.

Análisis por secciones:

- **Preparación:** $O(n \cdot m)$ (copia de la matriz).
- **Bucle exterior (días):** se ejecuta exactamente $\lceil n/k \rceil$ veces.
- **Dentro del bucle (por cada día):**
 - **Búsqueda del ancla:** $O(n)$ (encontrar el máximo en la columna `Total`).
 - **Cálculo de compatibilidad:** $O(n \cdot m)$ (recorre n tomas y compara m actores con `np.all`).
 - **Ordenación (`sort_values`):** $O(n \log n)$ (Timsort sobre las tomas restantes).
 - **Actualización (`drop`):** $O(n)$ (eliminar filas del DataFrame).

La complejidad es:

$$T(n) = O(n \cdot m) + \frac{n}{k} \cdot \left[\underbrace{n \cdot m}_{\text{Compatibilidad}} + \underbrace{n \log n}_{\text{Ordenación}} + \underbrace{2n}_{\text{Ancla y Drop}} \right]$$

Si distribuimos el término $\frac{n}{k}$:

$$T(n) = O(n \cdot m) + O\left(\frac{n^2 \cdot m}{k}\right) + O\left(\frac{n^2 \log n}{k}\right) + O\left(\frac{2n^2}{k}\right)$$

Resultado final (Big O):

$$O\left(\frac{n^2 \cdot (m + \log n)}{k}\right)$$

Además,

$$\frac{n}{k} \times (n \times m) = \frac{n^2 \cdot m}{k}$$

Si consideramos m del mismo orden que n , entonces el término dominante es $\frac{n^3}{k}$, y en ese caso hablamos de $O(n^3)$.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Búsqueda Aleatoria

Ya vimos que la Fuerza Bruta es inviable aquí, y por otra parte la voracidad no asegura la solución más óptima. Ya que el enunciado pide "diseñar un juego de datos de entrada aleatorios", hemos llegado a la conclusión de que usar una exploración aleatoria podría ser una pista en este caso, por lo tanto vamos a proceder a aplicar el algoritmo de búsqueda aleatoria.

La lógica a seguir será: en lugar buscar todas las combinaciones, lanzamos X intentos al azar y nos quedamos con el mejor de ellos. Al ser probabilístico y heruístico, no podemos asegurar tampoco que es la solución más óptima.

```
In [13]: def busqueda_aleatoria(intentos, data, max_tomas_dia, mostrar_info=True):
    # Calculamos en número de tomas
    total_tomas = len(data)
    # Calculamos días necesarios y redondeamos hacia arriba
    dias_necesarios = math.ceil(total_tomas / max_tomas_dia)

    if mostrar_info:
        # data.shape[1] nos da el numero de columnas (actores)
        print(f'Actores totales en la grabación: {data.shape[1]} ')
        print(f'Tomas a grabar: {total_tomas} ')
        print(f'Límite de tomas al día: {max_tomas_dia} -> Mínimo {dias_necesarios}')

    # --- Complejidad en área 1: Preparación [O(n)] ---

    # Crearemos una lista con los días del tipo [1,1,1, 2,2,2, ...]
    # Lista vacía para empezar
    distribucion_base = []

    # Se recorre cada día necesario
    for dia in range(1, dias_necesarios + 1):
        # Se añade el número del día actual "max_tomas_dia" veces
        for _ in range(max_tomas_dia):
            distribucion_base.append(dia)

    # Ajustamos al tamaño real de tomas por si acaso
    distribucion_base = distribucion_base[:total_tomas]

    # Hacemos un "for" de exploración
    coste_minimo = float('inf')
    mejor_plan = []

    # --- Complejidad en área 2: Bucle exploración [intentos (i)] ---

    for i in range(intentos):
        # --- Complejidad en área 3: Mezcla aleatoria [O(n)] ---

        # Desordenamos los días para probar una mezcla nueva
        random.shuffle(distribucion_base)
```

```

# --- Complejidad en área 4: coste de rodaje [O(n)] ---

# Calculamos el coste del shuffle actual
coste_actual = calcular_coste_rodaje(distribucion_base, data, verbos

# Si es mejor que lo anterior lo guardamos
if coste_actual < coste_minimo:
    coste_minimo = coste_actual

# --- Complejidad en área 5: copia de solución [O(n)] ---
mejor_plan = distribucion_base.copy()
if mostrar_info:
    print(f'Intento {i+1}: Nuevo mejor coste {coste_minimo}')

if mostrar_info:
    print(f'\nFinalizado. Coste final: {coste_minimo}')

return mejor_plan, coste_minimo

```

Aplica el algoritmo **Búsqueda Aleatoria** al juego de datos original

```

In [14]: # Cronómetro
         inicio_aleatoria = time.time()

# Ejecutamos la función y usamos 10000 intentos como ejemplo
plan_aleatorio, coste_busq_aleatoria_dataset_original = busqueda_aleatoria(1

# --- RESULTADOS ---
tiempo_busq_aleatoria_dataset_original = round(time.time() - inicio_aleatori
print(f'Estrategia Aleatoria: coste {coste_busq_aleatoria_dataset_original}

```

Actores totales en la grabación: 11

Tomas a grabar: 30

Límite de tomas al día: 6 -> Mínimo 5 días

Intento 1: Nuevo mejor coste 39

Intento 2: Nuevo mejor coste 35

Intento 113: Nuevo mejor coste 34

Finalizado. Coste final: 34

Estrategia Aleatoria: coste 34 en 3.96 segundos

Aplica el algoritmo **Búsqueda Aleatoria** al juego de datos aleatorios generado

```

In [15]: # Cronómetro
         inicio_aleatoria = time.time()

# Ejecutamos la función y usamos 1000 intentos como ejemplo
plan_aleatorio, coste_busq_aleatoria_dataset_aleatorio = busqueda_aleatoria(

# --- RESULTADOS ---

```

```
tiempo_busq_aleatoria_dataset_aleatorio = round(time.time() - inicio_aleatorio)
print(f'Estrategia Aleatoria: coste {coste_busq_aleatoria_dataset_aleatorio}
```

Actores totales en la grabación: 11

Tomas a grabar: 30

Límite de tomas al día: 6 → Mínimo 5 días

Intento 1: Nuevo mejor coste 49

Intento 2: Nuevo mejor coste 48

Intento 8: Nuevo mejor coste 47

Intento 46: Nuevo mejor coste 46

Intento 73: Nuevo mejor coste 44

Finalizado. Coste final: 44

Estrategia Aleatoria: coste 44 en 5.77 segundos

Variables:

- n : Número total de tomas.
- m : Número de actores (columnas).
- i : Número de intentos (iteraciones de búsqueda)

Análisis por secciones:

- **Preparación:** $O(n)$ (Crear la lista base y reordenarla).
- **Bucle exploración:** Se ejecuta i veces.
- **Dentro del bucle (por cada intento):**
 - **Mezcla:** $O(n)$ (se reordena la lista de tomas)
 - **Evaluación:** $O(n \cdot m)$ (Recorre las tomas y actores para calcular el coste).
 - **Actualización:** $O(n)$ (Copia el mejor plan encontrando hasta el momento).

La complejidad es:

$$T(n) = O(n) + i \cdot \left[\underbrace{n}_{\text{Shuffle}} + \underbrace{n \cdot m}_{\text{Evaluación}} + \underbrace{n}_{\text{Copia}} \right]$$

Si distribuimos el término i :

$$T(n) = O(n) + O(i \cdot n) + O(i \cdot n \cdot m) + O(i \cdot n)$$

Resultado Final (Big O):

$$\boxed{O(i \cdot n \cdot m)}$$

Esta complejidad es lineal respecto al número de intentos (i) y al número de tomas (n), multiplicada por la complejidad de calcular el coste de cada rodaje. La fórmula aproximada es: $O(i \cdot n \cdot m)$. Entonces si $n = m$ podríamos decir que la complejidad total es $O(i \times n^2)$

Búsqueda Local

Después de haber usado la búsqueda aleatoria, podemos proceder a utilizar la **búsqueda local**. Mientras la búsqueda aleatoria desperdicia recursos en zonas irrelevantes, la local optimiza el esfuerzo computacional al centrarse en **regiones prometedoras**

Búsqueda local con intercambio.

En este caso usaremos el operador de intercambio, es decir, creamos la vecindad generando las diferentes soluciones que pueden aparecer cuando se realiza el intercambio de 2 nodos.

```
In [16]: def intercambio(solucion_referencia, data, mostrar_info=True):
    # Guardamos la mejor configuración encontrada dentro de la vecindad actual
    mejor_vecino_global = []
    # Usamos infinito solo para asegurar será menor
    coste_minimo_vecindad = float('inf')

    # Se genera la Vecindad (Complejidad N^2)
    # En el primer bucle definimos el primer elemento a intercambiar
    for i in range(0, len(solucion_referencia) - 1):
        # En el segundo bucle se define el segundo elemento (siempre el de la derecha)
        for j in range(i + 1, len(solucion_referencia)):

            # Construcción de la solución candidata usando slicing y creando una copia
            vecino_candidato = (solucion_referencia[:i] + [solucion_referencia[i],
                                                             solucion_referencia[j]] +
                               solucion_referencia[i+1:j] + [solucion_referencia[i],
                                                             solucion_referencia[j]])

            # Calculamos el coste
            coste_candidato = calcular_coste_rodaje(vecino_candidato, data,

            # Si el coste del candidato es más bajo en la búsqueda entonces
            if coste_candidato < coste_minimo_vecindad:
                coste_minimo_vecindad = coste_candidato
                mejor_vecino_global = vecino_candidato

    # Mostramos los resultados
    if mostrar_info:
        print(f'Mejor vecino identificado con coste {coste_minimo_vecindad}:')

    # Retornamos el mejor plan encontrado en todo el entorno de la solución
    return mejor_vecino_global
```

```
In [17]: def busqueda_local(data, max_tomas_dia, func_generar_vecino=intercambio, mostrar_info=True):
    # Calculamos el número de tomas
    total_tomas = len(data)
    # Calculamos días necesarios y redondeamos hacia arriba
    dias_necesarios = math.ceil(total_tomas / max_tomas_dia) # Cálculo del número de días necesarios

    if mostrar_info:
        # data.shape[1] nos da el número de columnas (actores)
        print(f'Actores totales en la grabación: {data.shape[1]} ')
        print(f'Tomas a grabar: {total_tomas} ')
```

```

    print(f'Límite de tomas al día: {max_tomas_dia} -> Mínimo {dias_necesarios}')

    # Crearemos una lista con los días del tipo [1,1,1, 2,2,2, ...]
    # Lista vacía para empezar
    distribucion_base = []

    # Se recorre cada día necesario
    for dia in range(1, dias_necesarios + 1):
        # Se añade el número del día actual "max_tomas_dia" veces
        for _ in range(max_tomas_dia):
            distribucion_base.append(dia)

    # Ajustamos al tamaño real de tomas por si acaso
    distribucion_base = distribucion_base[:total_tomas]

    # Aplicamos aleatoriedad
    random.shuffle(distribucion_base)

    if mostrar_info:
        print(f'Punto de inicio generado: {distribucion_base}')

    # Establecemos el primer coste calculado como el mínimo a batir.  $O(n*m)$ 
    minimo_encontrado = calcular_coste_rodaje(distribucion_base, data, verbo=0)
    mejor_asignacion_local = distribucion_base

    num_iteracion = 0
    # Aquí empieza el bucle principal
    while True:
        num_iteracion += 1

        # Generamos una solución vecina dependiendo del operador elegido
        solucion_candidata = func_generar_vecino(mejor_asignacion_local, data, verbo=0)

        # Calculamos el coste del nuevo plan vecino
        coste_del_vecino = calcular_coste_rodaje(solucion_candidata, data, verbo=0)

        # Si se encuentra un vecino mejor se reasigna, es decir, con coste menor
        if coste_del_vecino < minimo_encontrado:
            minimo_encontrado = coste_del_vecino
            mejor_asignacion_local = solucion_candidata # Saltamos a la nueva asignación
        else:
            # Si no hay mejora, hemos llegado a un óptimo local
            if mostrar_info:
                print(f'Parada en la iteración {num_iteracion}, coste {minimo_encontrado}')

            return mejor_asignacion_local, minimo_encontrado

```

```
In [18]: # busqueda_local(data, 6, intercambio);
```

Aplica el algoritmo Búsqueda Local al juego de datos original

```
In [19]: # Cronómetro
inicio_local = time.time()
```

```
# Ejecutamos la función de búsqueda local
# Pasamos la función 'intercambio' como el generador de vecinos
plan_local, coste_busq_local_dataset_original = busqueda_local(data, max_ton

# --- RESULTADOS ---
tiempo_busq_local_dataset_original = round(time.time() - inicio_local, 2)
print(f'\nEstrategia Búsqueda Local: coste {coste_busq_local_dataset_origina
```

Actores totales en la grabación: 11

Tomas a grabar: 30

Límite de tomas al día: 6 -> Mínimo 5 días

```
Punto de inicio generado: [3, 4, 2, 3, 3, 4, 5, 5, 1, 4, 2, 2, 2, 2, 4, 4,
5, 4, 1, 2, 1, 1, 1, 3, 3, 5, 5, 1, 5, 3]
Mejor vecino identificado con coste 37: [4, 4, 2, 3, 3, 4, 5, 5, 1, 4, 2, 2,
2, 2, 3, 4, 5, 4, 1, 2, 1, 1, 1, 3, 3, 5, 5, 1, 5, 3]
Mejor vecino identificado con coste 34: [4, 4, 2, 3, 3, 4, 5, 5, 1, 4, 2, 2,
2, 2, 3, 3, 5, 4, 1, 2, 1, 1, 1, 4, 3, 5, 5, 1, 5, 3]
Mejor vecino identificado con coste 33: [5, 4, 2, 3, 3, 4, 5, 5, 1, 4, 2, 2,
2, 2, 3, 3, 5, 4, 1, 2, 1, 1, 1, 4, 3, 4, 5, 1, 5, 3]
Mejor vecino identificado con coste 32: [5, 4, 1, 3, 3, 4, 5, 5, 1, 4, 2, 2,
2, 2, 3, 3, 5, 4, 1, 2, 2, 1, 1, 4, 3, 4, 5, 1, 5, 3]
Mejor vecino identificado con coste 31: [5, 4, 5, 3, 3, 4, 5, 5, 1, 4, 2, 2,
2, 2, 3, 3, 1, 4, 1, 2, 2, 1, 1, 4, 3, 4, 5, 1, 5, 3]
Mejor vecino identificado con coste 30: [4, 4, 5, 3, 3, 5, 5, 5, 1, 4, 2, 2,
2, 2, 3, 3, 1, 4, 1, 2, 2, 1, 1, 4, 3, 4, 5, 1, 5, 3]
Mejor vecino identificado con coste 30: [4, 4, 5, 3, 3, 5, 5, 5, 1, 4, 2, 2,
2, 2, 3, 3, 1, 4, 1, 2, 2, 1, 1, 4, 3, 4, 5, 1, 5, 3]
Parada en la iteración 7, coste 30: [4, 4, 5, 3, 3, 5, 5, 5, 1, 4, 2, 2, 2,
2, 3, 3, 1, 4, 1, 2, 2, 1, 1, 4, 3, 4, 5, 1, 5, 3]
```

Estrategia Búsqueda Local: coste 30 en 12.8 segundos

Aplica el algoritmo Búsqueda Local al juego de datos aleatorios generado

```
In [20]: # Cronómetro
inicio_local_df = time.time()

# Ejecutamos la función sobre el DataFrame aleatorio
plan_local_df, coste_busq_local_dataset_aleatorio = busqueda_local(df_aleato

# --- RESULTADOS ---
tiempo_busq_local_dataset_aleatorio = round(time.time() - inicio_local_df, 2)
print(f'\nEstrategia Búsqueda Local (DF Aleatorio): coste {coste_busq_local_
```

Actores totales en la grabación: 11
Tomas a grabar: 30
Límite de tomas al día: 6 -> Mínimo 5 días

Punto de inicio generado: [1, 4, 5, 2, 2, 3, 4, 3, 3, 1, 4, 5, 1, 4, 5, 2, 3, 1, 5, 5, 3, 2, 3, 5, 4, 4, 2, 1, 1, 2]
Mejor vecino identificado con coste 46: [1, 4, 5, 2, 2, 3, 4, 2, 3, 1, 4, 5, 1, 4, 5, 2, 3, 1, 5, 5, 3, 3, 3, 5, 4, 4, 2, 1, 1, 2]
Mejor vecino identificado con coste 44: [1, 4, 5, 2, 2, 3, 4, 2, 3, 4, 4, 5, 1, 1, 5, 2, 3, 1, 5, 5, 3, 3, 3, 5, 4, 4, 2, 1, 1, 2]
Mejor vecino identificado con coste 43: [1, 4, 5, 2, 2, 3, 4, 2, 3, 4, 4, 2, 1, 1, 5, 2, 3, 1, 5, 5, 3, 3, 3, 5, 4, 4, 5, 1, 1, 2]
Mejor vecino identificado con coste 42: [1, 4, 5, 2, 2, 3, 4, 2, 3, 4, 4, 2, 1, 1, 5, 2, 3, 1, 5, 5, 3, 3, 2, 5, 4, 4, 5, 1, 1, 3]
Mejor vecino identificado con coste 42: [5, 4, 1, 2, 2, 3, 4, 2, 3, 4, 4, 2, 1, 1, 5, 2, 3, 1, 5, 5, 3, 3, 2, 5, 4, 4, 5, 1, 1, 3]
Parada en la iteración 5, coste 42: [1, 4, 5, 2, 2, 3, 4, 2, 3, 4, 4, 2, 1, 1, 5, 2, 3, 1, 5, 5, 3, 3, 2, 5, 4, 4, 5, 1, 1, 3]

Estrategia Búsqueda Local (DF Aleatorio): coste 42 en 10.09 segundos

Variables:

- n : Número total de tomas.
- m : Número de actores.
- V : Tamaño de la vecindad (todos los pares posibles de intercambio).

Análisis por secciones:

- **Generación de Vecindad (V):** Para intercambiar 2 nodos de una lista de tamaño n , existen $\frac{n(n-1)}{2}$ combinaciones posibles. Esto es $O(n^2)$.
- **Bucle de exploración de la vecindad:** Se ejecuta $O(n^2)$ veces (una por cada posible intercambio).
- **Dentro del bucle (por cada vecino):**
 - **Intercambio:** $O(n)$
 - **Evaluación:** $O(n \cdot m)$ (Tu función calcular_coste_rodaje que recorre tomas y actores).
 - **Comparación y Actualización:** $O(n)$ (Guardar el mejor vecino encontrado hasta el momento).
- La complejidad es:

$$T(n) = \underbrace{n^2}_{\text{Vecindad}} \cdot \left[\underbrace{n}_{\text{Swap}} + \underbrace{n \cdot m}_{\text{Evaluación}} + \underbrace{n}_{\text{Actualización}} \right]$$

- Si distribuimos los términos:

$$T(n) = O(n^3) + O(n^3 \cdot m) + O(n^3)$$

- Resultado Final (Big O):

$$O(n^3 \cdot m) = \boxed{O(n^4)}$$

Recocido Simulado

A continuación intentaremos usar el algoritmo de Recocido Simulado, ya que después de probar con la búsqueda local y el algoritmo voraz parece ser que se quedan óptimos locales. Al principio, el recocido aceptará planes de rodaje que son un poco más caros en actores, pero después podrá explorar combinaciones de tomas que a simple vista parecen peores, pero que acaban llevándonos a una mejor solución.

```
In [22]: #Generador de 1 solución vecina 2-opt 100% aleatoria (intercambiar 2 nodos)
#Mejorable eligiendo otra forma de elegir una vecina.
def genera_vecina_aleatorio(solucion):

    #Se eligen dos nodos aleatoriamente
    i,j = sorted(random.sample( range(1,len(solucion)) , 2))

    #Devuelve una nueva solución pero intercambiando los dos nodos elegidos al
    return solucion[:i] + [solucion[j]] + solucion[i+1:j] + [solucion[i]] + sc

#Funcion de probabilidad para aceptar peores soluciones
def probabilidad(T,d):
    if random.random() < math.exp( -1*d / T) :
        return True
    else:
        return False

#Funcion de descenso de temperatura
def bajar_temperatura(T):
    return T*0.99
```

```
In [49]: def recocido_simulado_rodaje(data, temp_inicial, max_tomas_dia):
    n_tomas = len(data)

    # Tenemos que crear una función interna para traducir secuencia a plan y
    def medir_coste(secuencia_de_las_tomas):
        plan = [0] * n_tomas
        for i, toma in enumerate(secuencia_de_las_tomas):
            plan[toma] = (i // max_tomas_dia) + 1
        # Usamos tu función original de coste
        return calcular_coste_rodaje(plan, data, verbose=False)

    # 1. Solución inicial aleatoria
    solucion_referencia = list(range(n_tomas))
    random.shuffle(solucion_referencia)
    distancia_referencia = medir_coste(solucion_referencia)

    mejor_solucion = list(solucion_referencia)
    mejor_distancia = distancia_referencia
```

```

T = temp_inicial
iteracion = 0

while T > 0.0001:
    iteracion += 1

    # 2. Generar vecina y medir
    vecina = genera_vecina_aleatorio(solucion_referencia)
    distancia_vecina = medir_coste(vecina)

    delta = distancia_vecina - distancia_referencia

    # 3. Actualizar mejor global
    if distancia_vecina < mejor_distancia:
        mejor_solucion = list(vecina)
        mejor_distancia = distancia_vecina

    # 4. Decisión de movimiento (Aceptación de Metropolis)
    if delta < 0 or probabilidad(T, delta):
        solucion_referencia = list(vecina)
        distancia_referencia = distancia_vecina

    # 5. Enfriamiento
    T = bajar_temperatura(T)

    if iteracion % 500 == 0:
        print(f"Iteración {iteracion} | T: {T:.2f} | Mejor Coste: {mejor_distancia}")

return mejor_solucion, mejor_distancia

```

```

In [50]: TEMP_INICIAL = 1000
max_tomas_dia = 6

mejor_plan_rs, mejor_coste_rs = recocido_simulado_rodaje(data, TEMP_INICIAL,
print(f"\nResultado Recocido Simulado: {mejor_coste_rs} actores totales.")

```

```

Iteración 500 | T: 6.57 | Mejor Coste: 34
Iteración 1000 | T: 0.04 | Mejor Coste: 30
Iteración 1500 | T: 0.00 | Mejor Coste: 29

```

Resultado Recocido Simulado: 29 actores totales.

Aplica el algoritmo Recocido Simulado al juego de datos original

```

In [51]: # Configuración recocido
TEMP_INICIAL = 5000
max_tomas_dia = 6

# Cronómetro
inicio_recocido = time.time()

# Ejecutamos la función de recocido simulado
plan_recocido, coste_recocido_dataset_original = recocido_simulado_rodaje(

```

```

    data,
    temp_inicial=TEMP_INICIAL,
    max_tomas_dia=max_tomas_dia
)

# Resultados
tiempo_recocido_dataset_original = round(time.time() - inicio_recocido, 2)

print(f'\nAlgoritmo de Recocido Simulado: coste {coste_recocido_dataset_orig

```

Iteración 500 | T: 32.85 | Mejor Coste: 34
 Iteración 1000 | T: 0.22 | Mejor Coste: 32
 Iteración 1500 | T: 0.00 | Mejor Coste: 30

Estrategia Recocido Simulado: coste 29 en 6.61 segundos

Aplica el algoritmo Recocido Simulado al juego de datos aleatorio

```

In [57]: # Configuración recocido
TEMP_INICIAL = 5000
max_tomas_dia = 6

# Cronómetro
inicio_recocido = time.time()

# Ejecutamos la función de recocido simulado
plan_recocido, coste_recocido_dataset_aleatorio = recocido_simulado_rodaje(
    df_aleatorio,
    temp_inicial=TEMP_INICIAL,
    max_tomas_dia=max_tomas_dia
)

# Resultado
tiempo_recocido_dataset_aleatorio = round(time.time() - inicio_recocido, 2)

print(f'\nAlgoritmo de Recocido Simulado: coste {coste_recocido_dataset_alea

```

Iteración 500 | T: 32.85 | Mejor Coste: 44
 Iteración 1000 | T: 0.22 | Mejor Coste: 42
 Iteración 1500 | T: 0.00 | Mejor Coste: 39

Algoritmo de Recocido Simulado: coste 38 en 6.59 segundos

Para el algoritmo de **Recocido Simulado** tenemos:

Variables:

- n : Número total de tomas (30).
- m : Número de actores (11).
- I : Número total de iteraciones (determinado por $T_{inicial}$, T_{final} y el factor de enfriamiento).
- k : Límite de tomas por día (6).

Análisis por secciones:

- **Bucle de Enfriamiento:** Se ejecuta I veces.
- **Dentro del bucle (por cada iteración):**
 - **Generar Vecino (Intercambio):** $O(n)$. Se seleccionan dos posiciones aleatorias y se intercambian.
 - **Traducción de Secuencia a Plan:** $O(n)$. Se cada toma a su día correspondiente dividiendo la secuencia en bloques de tamaño k .
 - **Evaluación del Coste (calcular_coste_rodaje):** $O(n \cdot m)$. Se verifica la asistencia de cada actor a las tomas de cada día.
 - **Criterio de Metrópolis:** $O(1)$. Un cálculo exponencial y una comparación lógica.

Complejidad Total: La complejidad es el producto de las iteraciones por el trabajo de evaluación interna:

$$T(n) = I \cdot \left[\underbrace{n}_{\text{Intercambio}} + \underbrace{n}_{\text{Traducción}} + \underbrace{n \cdot m}_{\text{Evaluación}} \right]$$

Resultado Final (Big O): Simplificando los términos y manteniendo el dominante dentro del bucle: m

$$O(I \cdot n \cdot m)$$

Tabla comparativa con resultados

```
In [60]: from IPython.display import display, Markdown

tabla_resultados = f"""
### Tabla Comparativa de Resultados

| Modelo | Solución (Coste) | Tiempo (s) | Tomas | Máx. por día | Dataset |
| :--- | :---: | :---: | :---: | :---: | :---: |
| **Fuerza Bruta** | **{mejor_coste_FB_dataset_original}** | **{tiempo_total_FB_dataset_original}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Fuerza Bruta** | **{mejor_coste_FB_dataset_aleatorio}** | **{tiempo_total_FB_dataset_aleatorio}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Algoritmo Voraz** | **{coste_voraz_dataset_original}** | **{tiempo_voraz_dataset_original}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Algoritmo Voraz** | **{coste_voraz_dataset_aleatorio}** | **{tiempo_voraz_dataset_aleatorio}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Búsqueda Aleatoria** | **{coste_busq_aleatoria_dataset_original}** | **{tiempo_busq_aleatoria_dataset_original}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Búsqueda Aleatoria** | **{coste_busq_aleatoria_dataset_aleatorio}** | **{tiempo_busq_aleatoria_dataset_aleatorio}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Búsqueda Local** | **{coste_busq_local_dataset_original}** | **{tiempo_busq_local_dataset_original}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Búsqueda Local** | **{coste_busq_local_dataset_aleatorio}** | **{tiempo_busq_local_dataset_aleatorio}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Recocido Simulado** | **{coste_recocido_dataset_original}** | **{tiempo_recocido_dataset_original}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |
| **Recocido Simulado** | **{coste_recocido_dataset_aleatorio}** | **{tiempo_recocido_dataset_aleatorio}** | **{tomas_max}** | **{max_por_dia}** | **{dataset}** |

"""
```



```
display(Markdown(tabla_resultados))
```

Tabla Comparativa de Resultados

Modelo	Solución (Coste)	Tiempo (s)	Tomas	Máx. por día	Dataset
Fuerza Bruta	19	108.96	8	2	Original
Fuerza Bruta	20	132.83	8	2	Aleatorio
Algoritmo Voraz	35	0.11	30	6	Original
Algoritmo Voraz	44	0.11	30	6	Aleatorio
Búsqueda Aleatoria	34	3.96	30	6	Original
Búsqueda Aleatoria	44	5.77	30	6	Aleatorio
Búsqueda Local	30	10.09	30	6	Original
Búsqueda Local	42	10.09	30	6	Aleatorio
Recocido Simulado	29	6.61	30	6	Original
Recocido Simulado	38	6.59	30	6	Aleatorio

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

- Reyero Díez, R. (2024). Algoritmos de optimización. Universidad Internacional de Valencia.
- Universidad Internacional de Valencia. (2024). Guía de Actividades: Actividades Guiadas y Trabajo Práctico de Algoritmos de Optimización
- Numpy: <https://numpy.org/doc/>
- Time: <https://docs.python.org/3/library/time.html>
- Pandas: <https://pandas.pydata.org/docs/>

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Propuestas de mejora para el sistema de doblaje

Para dar el siguiente paso en este proyecto, en caso de que el número de tomas o las restricciones aumente considerablemente, entonces tendremos que mirar otros algoritmos.

Metaheurísticas

Ahora mismo, si un algoritmo encuentra una buena solución, entonces ya no continua. Por lo que necesitamos técnicas que **no** se conformen con lo primero que encuentran:

- **Búsqueda Tabú:** Básicamente, le damos al algoritmo un 'diario' para que recuerde qué soluciones ya probó y no vuelva a caer en los mismos errores (o en los mismos 'mínimos locales').
- **Recocido Simulado:** Al principio le dejamos probar todo tipo de soluciones (donde puede incluso empeorar) para explorar todo el terreno, y poco a poco hacemos que se enfoque en la mejor opción encontrada. (NOTA: EN ESTA SEGUNDA ENTREGA SE HA USADO EL ALGORITMO; Y SE HA CONFIRMADO QUE ERA UNA BUENA OPCIÓN A UTILIZAR)

Algoritmos Poblacionales

Si el problema crece demasiado, sería mejor probar muchas veces a la vez, en vez de buscar una solución. Es mejor probar muchas a la vez:

- **Algoritmos Genéticos:** Creamos un grupo de posibles horarios. Hacemos que los mejores horarios combinen sus partes buenas y metemos cambios aleatorios para ver si encontramos algo mejor.
- **Colonia de Hormigas:** Vamos probando probando combinaciones y dejamos pistas las agrupaciones de tomas que resultan ser más baratas.

Hibridación

Otra opción sería no usar una sola técnica, sino mezclar las partes de buena de cada una:

- Podemos usar nuestro **algoritmo voraz** actual para tener una solución decente rápida, y luego aplicamos **GRASP** (aleatoriedad) para encontrar la verdadera mejor opción.

Modificarlo para casos reales o nuevos modelos.

El modelo actual para alguien experto, puede ser considerado simple. Y puedes que no se capaz de adaptarse a la vida real. A veces esta adaptación a la vida real podría llevar a caminos más complejos pero efectivos:

- **Varios o diferentes objetivos:** En este caso importa el dinero, pero en otro podría ser el terminar el proyecto lo antes posible.

- **Cambios de última hora:** El algoritmo debería poder recalcular todo si un actor se enferma de repente (penasando en un modelo más dinámico).

Escalabilidad

Por último, habría que tener en cuenta la duración de esos algoritmos nuevos. No sirve de nada una solución perfecta si tarda tres días en calcularse.

Clasificación Integral de Algoritmos de Optimización

Esta tabla la creamos para tener un visión general de los diferentes algoritmos. Esta nos ha ayudado a a decidir que tipos de algoritmos podrían haberse utilizado en el proyecto.

Algoritmo	Categoría	Forma de Calcular	Calidad de Solución	Características Principales	Aplicacion Comunes
Simplex	Programación Matemática	Determinista	Exacto	Se desplaza por la frontera de la región factible hasta encontrar el óptimo matemático.	Problemas transporte, producción
Ramificación y Poda	Búsqueda Informada	Determinista	Exacto	Divide el problema en subproblemas y descarta ramas que no pueden superar al mejor resultado hallado.	Agente Viaj (TSP) y pro NP-comple
Programación Dinámica	Programación Dinámica	Determinista	Exacto	Divide el problema en subproblemas superpuestos y almacena sus soluciones en tablas para evitar cálculos repetidos.	Problema d mochila y r coste mínir
Vuelta Atrás (Backtracking)	Búsqueda Sistemática	Determinista	Exacto	Explora un árbol de expansión de manera exhaustiva y retrocede cuando una rama deja de ser prometedora.	Problema d reinas y lab
Ordenación (p.ej. Merge Sort)	Algoritmos de Ordenación	Determinista	Exacto	Sigue pasos finitos y precisos para organizar una lista bajo un criterio estricto.	Gestión de datos y list
Algoritmos Voraces (Greedy)	Técnica de Diseño	Determinista	Heurístico / Aproximado	Elige en cada etapa la decisión óptima local sin considerar las consecuencias	Árboles de recubrimier (Prim/Krust

Algoritmo	Categoría	Forma de Calcular	Calidad de Solución	Características Principales	Aplicaciones Comunes
				futuras; no garantiza el óptimo global.	cambio de monedas.
Multiarranque (Multi-start)	Metaheurística Constructiva	Probabilista / Determinista	Heurístico	Repite el proceso de generar una solución inicial y aplicar búsqueda local para explorar diversas regiones y evitar mínimos locales.	Optimización y problemas de rutas complejas.
Búsqueda Aleatoria	Metaheurística	Probabilista	Heurístico	Genera soluciones totalmente al azar en cada iteración y devuelve la mejor encontrada.	Exploración en espacios desconocidos.
Búsqueda Local	Metaheurística	Determinista / Probabilista	Heurístico	Parte de una solución inicial y busca mejorarla explorando su vecindad inmediata; tiene riesgo de estancarse en óptimos locales.	Refinamiento de soluciones y problemas de optimización.
GRASP	Metaheurística	Probabilista	Heurístico	Combina una fase constructiva voraz-aleatoria con una fase de mejora mediante búsqueda local.	Problemas logísticos y recubrimiento de conjuntos.
Búsqueda Tabú	Metaheurística	Determinista / Probabilista	Heurístico	Utiliza una memoria adaptativa (lista tabú) para prohibir movimientos recientes y forzar la exploración de nuevas áreas.	Programación de horarios (scheduling) y secuenciación industrial.
Recocido Simulado	Metaheurística	Probabilista	Heurístico	Acepta soluciones peores con una probabilidad que disminuye con el tiempo, imitando el enfriamiento de metales.	Diseño de circuitos integrados y metalurgia computacional.
Algoritmos Genéticos	Poblacional	Probabilista	Heurístico	Se basan en mecanismos de selección natural, cruce y mutación aplicados a una	Bioinformática y optimización de funciones complejas.

Algoritmo	Categoría	Forma de Calcular	Calidad de Solución	Características Principales	Aplicaciones Comunes
Colonias de Hormigas (ACO)	Poblacional	Probabilista	Heurístico	población de soluciones.	Optimización de redes de transporte y telecomunicaciones.
				Múltiples agentes depositan feromonas en las rutas para guiar probabilísticamente hacia la mejor solución compartida.	
Descenso del Gradiente	Optimización Continua	Determinista	Exacto (en funciones convexas)	Algoritmo iterativo que minimiza una función continua siguiendo la dirección opuesta al gradiente.	Entrenamiento de Redes Neuronales, Machine Learning.
Dijkstra	Búsqueda en Grafos	Determinista	Exacto	Encuentra el camino más corto desde un nodo origen al resto, siempre que no haya pesos negativos.	Navegadores, protocolos de red.